

Week 6 — Spark Architecture & Efficient Data Processing

Celebal Technologies — Data Engineer Internship

Palak Rathore

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

The Driver is the process that runs the application's `main()` function. It builds the logical plan (DAG) from the transformations in the code, negotiates resources with the Cluster Manager, splits the work into tasks, schedules those tasks onto executors, and collects final results back for actions like `collect()` or `count()`.

The Cluster Manager (Standalone, YARN, Kubernetes, or Mesos) is responsible for allocating resources across the cluster — it decides which worker nodes the application's executors run on and how much CPU/memory each gets. It does not process data itself.

Executors are worker processes launched on cluster nodes for the lifetime of the application. They execute the tasks assigned by the Driver, store data in memory or on disk for caching and shuffles, and report status and results back to the Driver.

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Transformations (`select`, `filter`, `withColumn`, `groupBy`, `join`, etc.) are not executed when they're written — Spark only records them as a lineage graph (DAG). Execution is deferred until an action (`count()`, `show()`, `collect()`, `write()`) is called.

This deferral lets Spark's Catalyst optimizer see the entire chain of operations at once before running anything, so it can:

- Combine multiple transformations into fewer physical stages.
- Push filters and column selection down as early as possible (predicate/projection pushdown) — sometimes right into the file scan.
- Skip computing branches of the DAG whose output is never actually needed.

The net effect is that a long chain of transformations on a huge dataset costs one optimized pass instead of many separate, unoptimized ones.

Q3. Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and `inferSchema` is enabled.

```
df = (spark.read
      .option("header", "true")
      .option("inferSchema", "true")
      .csv("data/source.csv"))
```

Note: this project's own pipeline deliberately avoids `inferSchema` in favor of an explicit `StructType` schema — `inferSchema` forces an extra pass over the file just to guess types, which is wasted cost on large files. Q3 is answered as literally asked.

Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV is row-based and stores every field of every row as plain, uncompressed text, one row after another. To read even a single column, Spark must read and parse every row in full.

Parquet is columnar: values from the same column are stored together, encoded with type-aware, compressed representations, and grouped into row-groups that carry min/max statistics per column.

This matters for performance in three ways: (1) queries that need only a few columns read far less data (column pruning), (2) filters can skip entire row-groups using the stored statistics without decompressing them (predicate pushdown), and (3) columnar compression is far more effective than row-based compression since similar values sit next to each other. In this project's own run, the same dataset was about 4x smaller as Parquet than as CSV on disk.

Q5. Given a `DataFrame df`, write a query to select the columns `product_id` and `price` where the category is 'Electronics'.

```
result = df.select("product_id", "price") \
            .filter(df.category == "Electronics")
```

Q6. Write the code to "revise" a `DataFrame` by renaming the column `old_name` to `new_name` and casting the `price` column from a `String` to a `Double`.

```
from pyspark.sql.types import DoubleType

df_revised = (df
              .withColumnRenamed("old_name", "new_name")
              .withColumn("price", df["price"].cast(DoubleType())))
```

Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Spark's `DataFrames`/`RDDs` are immutable, and every one of them remembers the exact sequence of transformations (the lineage) that produced it from the original source data. Spark never needs to replicate data purely for fault tolerance because of this.

If an executor holding some partition of data fails, the Driver simply looks at that partition's lineage and re-schedules the same transformations on a different, healthy executor to recompute just that lost partition — not the whole job. This is far cheaper than the replication-based fault tolerance used by systems without a recorded lineage graph.

Q8. Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

```
result = df_orders.filter(
    (df_orders.status == "Completed") & (df_orders.amount > 1000)
)
```

Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Predicate pushdown means Spark passes a query's filter conditions down to the file-reading layer instead of reading everything first and filtering afterward in memory.

Parquet stores per-column min/max statistics for each row-group in its footer metadata. When a filter like `sales > 500` is pushed down, Spark checks those statistics before touching the actual column data — any row-group whose max is below 500 is skipped entirely, never decompressed or loaded into memory. In this project's own physical plan output, the scan node showed `PushedFilters: [IsNotNull(Sales), GreaterThan(Sales,500.0)]` directly on the Parquet scan, confirming the filter ran at the storage layer rather than after a full read.

Q10. Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
df = df.withColumn("final_price", df["base_price"] * 1.18)
```

Q11. What is the difference between Transformations and Actions? Provide two examples of each.

Transformations describe how to derive a new DataFrame/RDD from an existing one. They are lazy — Spark only records them in the DAG and does not execute anything immediately. Examples: `select()`, `filter()`, `withColumn()`, `groupBy()`, `join()`, `map()`.

Actions trigger the actual execution of the accumulated DAG and return a result to the Driver or write output to storage. Examples: `count()`, `show()`, `collect()`, `write()`, `first()`.

Q12. Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where `user_id` is null, and save the result as a CSV at "path/to/output".

```
df = spark.read.parquet("path/to/input")
df_clean = df.filter(df.user_id.isNotNull())
df_clean.write.mode("overwrite").option("header", "true").csv("path/to/output")
```

Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

In Client Mode, the Driver runs on the machine that submitted the application — outside the cluster (e.g. a developer's laptop or an edge/gateway node). It communicates with the cluster's executors over

the network. This is convenient for interactive work like notebooks and debugging, but if the submitting machine disconnects or crashes, the Driver — and the whole application — dies with it.

In Cluster Mode, the Driver itself is launched as a process inside the cluster, managed by the Cluster Manager alongside the executors. The submitting machine can disconnect once the job is launched without killing the application. This is the preferred mode for unattended, production batch jobs.

Q14. Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
result = df.filter(
    (df.region == "North") | (df.priority == "High")
)
```

Q15. When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

`.collect()` pulls every single row of the DataFrame back across the network into the Driver's memory, as a local Python/Scala list. On a multi-terabyte dataset this means the entire dataset — not just a preview — has to fit in the memory of one single machine (the Driver), which will almost certainly cause an out-of-memory crash and take down the whole application.

`.show(5)` instead asks the executors to compute only enough partitions to produce 5 rows, and only those 5 rows are ever sent back to the Driver for display. It gives the same ability to sanity-check the data's shape and values, at a cost that stays constant regardless of how large the underlying dataset is.